CS-107 Lecture 7
Encoding and Storage

Troy Astarte

Megan Venn-Wycherley

Recap: real numbers in binary

- **Bits after the radix point** are halves, quarters, eighths: 11.101 is 3.625.
- **Convert a fraction** by multiplying by 2 and reading the whole parts in order.
- **Some fractions never end in binary:** 0.3 repeats, so it is stored rounded.
- **Floating point** keeps a sign, a mantissa and an exponent, standardised as IEEE 754.

Recap: representing text

- **Every character needs its own pattern,** so we agree a list and number it.
- **ASCII uses 7 bits:** 128 characters, including the non-printing control codes.
- **"0", 0 and NUL are three different things,** even though we say zero for all of them.
- **Unicode extends the idea:** the first 256 match extended ASCII, and UTF-16 reaches 65,536 characters.

Recap: colour, images and sound

- **Colour:** one byte each for red, green and blue gives 16.7 million shades.
- **Raster images** store a value per pixel, so size grows with resolution.
- **Vector images** store the maths, so they scale cleanly but suit diagrams, not photographs.
- **Sound is sampled:** more samples a second and more bits per sample mean better quality and bigger files.

Goals for Today

- Why we compress: storage and transmission
- Keyword and run-length encoding
- Lossless against lossy
- Variable-length codes and Huffman trees

Data Compression

Representing data and storing it

- All of the *things* we've discussed so far are represented as binary values on the machine.

- Some of these representations can be quite large:
  - One RGB colour requires us to store 24 bits (8 bits for R, G, and B).
  - A High-def image has a resolution of 1920 x 1080, or 2,073,600 pixels.
  - That would be just under 50 million bits to store a single picture.
  - Works out to roughly 6 mega-bytes.

Data Compression

- **Data compression**
  - Reduction in the amount of space needed to store a piece of data or the bandwidth to transmit it
- **Compression ratio**
  - The size of the compressed data divided by the size of the original data

$$\text{compression_ratio} = \frac{length(\text{compressed})}{length(\text{original})}$$

What does a small ratio mean? A large ratio?

Data Compression

A data compression technique can be:

lossless, which means the data can be retrieved without any loss of original information

lossy, which means some information may lost in the process of compression

When might you want one over the other?

Data Compression

- The examples we will use in this lecture will predominantly focus on text data, however remember that all of this information is represented on the machine as **bit strings**.

- This means you should think about how these methods may apply in other situations (i.e. different types of data rather than text), and how they may work on the bit string level.

Keyword Encoding

- Repeating single values may not be very common.
- What about finding frequently used patterns and replacing those with a single character?
- All we would need is a list of the common patterns and the single character that they were replaced with.

Keyword Encoding Example

The shop sells fruit and bread and drinks and clothes and vegetables

"and" is used a lot. If we encode the keyword *and* by & we get:

The shop sells fruit & bread & drinks & clothes & vegetables.

The original message has 69 characters and the compressed has 61 characters.
The compression rate is 61/69 = ~0.88

Keyword Encoding

- To perform encoding and decoding we need the mapping table. This can often be document or application specific.
- In our previous example, we would need to record that "and" is replaced with "&" and back.
- Obviously storing this table takes up space too, so this can affect how effective the compression is. (we'll ignore this in our calculations for the moment)

Run-Length Encoding

- In some kinds of files, a single value may be repeated many times in a long sequence.
- Can you think of any example scenarios?
- Why repeat these values, when we can have them once and then say how many times to repeat it. We can encode the **run length** of the value.

Run-Length Encoding

- Replace the repeated sequence with:
  - A flag character (such as * or \)
  - The value to repeat
  - The number of times to repeat it.



- Only need to do this with a **sequence of more than 3** values

- This may not happen to often in natural text, but it could happen quite often with binary values!

Run-Length Encoding

Example

Original text	aaaaabbbccccccc+!!!!!!
Encoded text	*a5bbb*c7+*!6

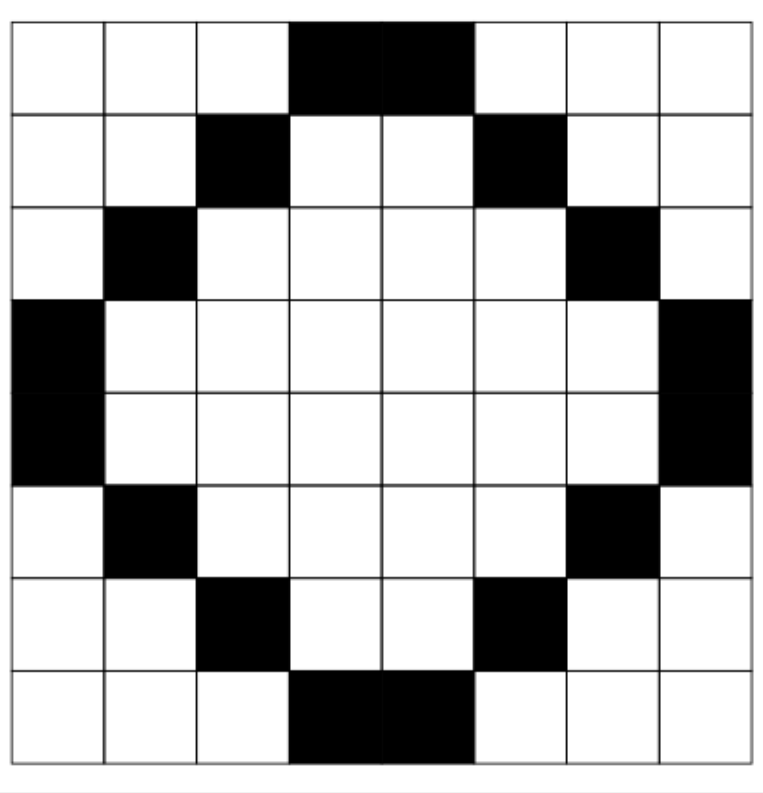
Compression ratio is $\frac{13}{22} = 0.59$.

Example

Encoded text	a*b5*c4de!
Decoded text	abbbbbccccde!

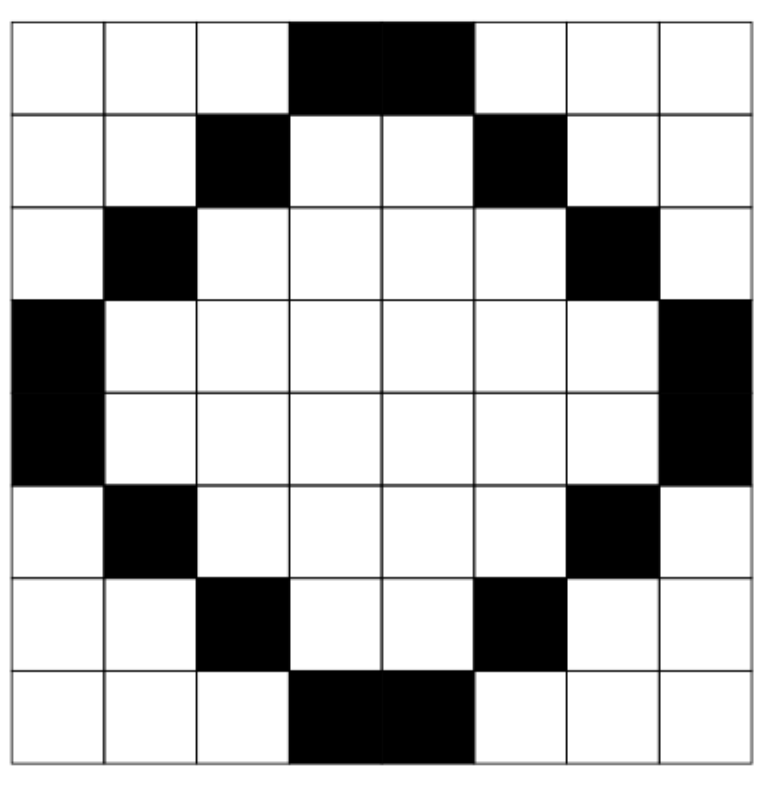
Consider the following

- Image is 8x8 pixels
- Each pixel either black (1) or white
  - Bit depth of 1
  - Need 64 bits of storage



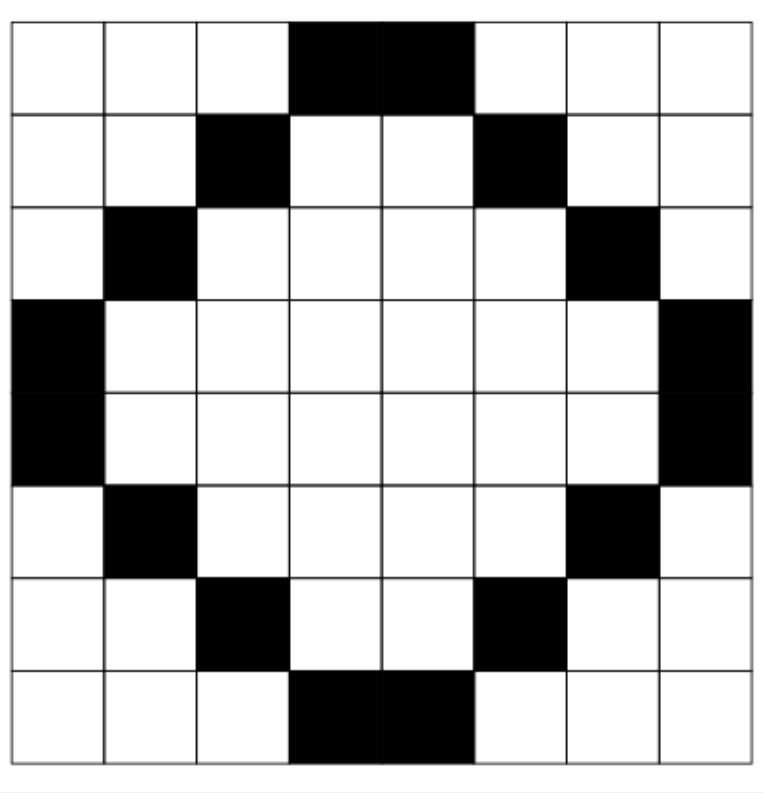
Consider the following

- In binary, represented as:
  - 00011000
  - 00100100
  - 01000010
  - 10000001
  - 10000001
  - 01000010
  - 00100100
  - 00011000



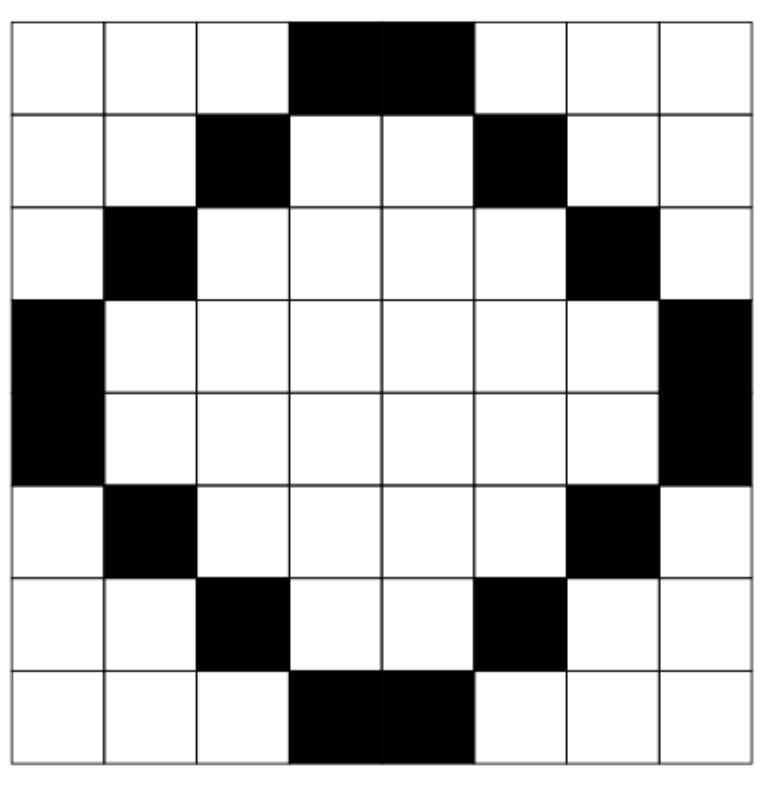
Compressing the image

- This largely composed of ‘runs’ of 0s, in solitary) 1s.
- We could encode the whole image as a -runs’ (how many zeros in a row)



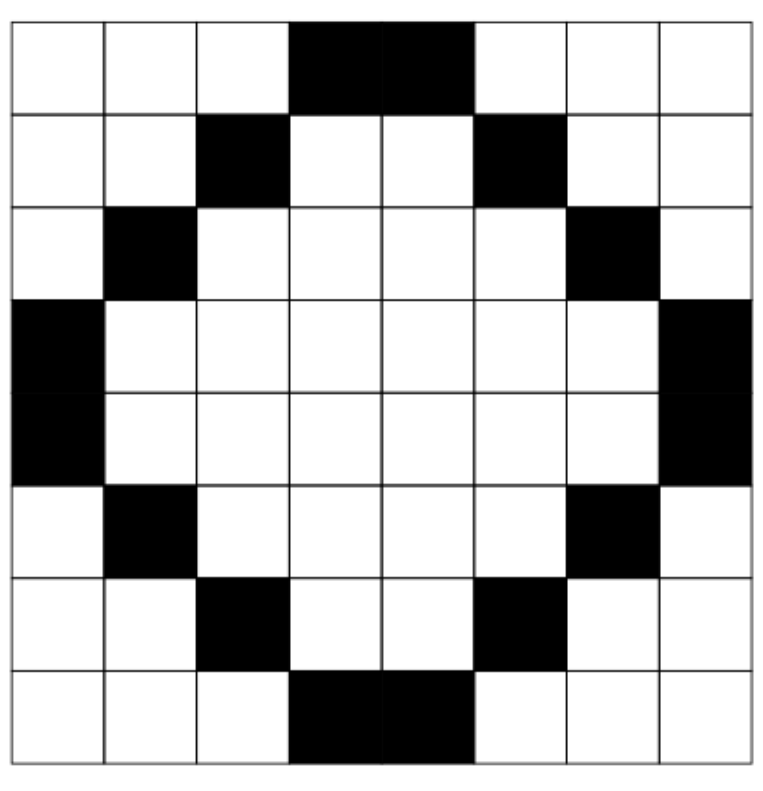
Consider the following

- How many zero-runs?
  - 00011000
  - 00100100
  - 01000010
  - 10000001
  - 10000001
  - 01000010
  - 00100100
  - 00011000



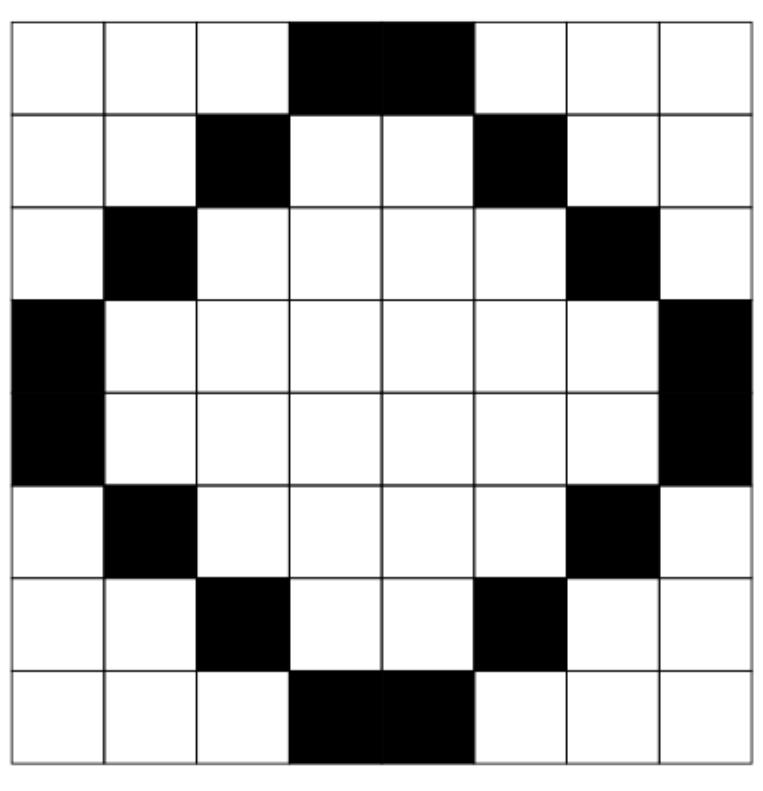
Consider the following

- How many zero-runs?
  - 00011000 3 0
  - 00100100
  - 01000010
  - 10000001
  - 10000001
  - 01000010
  - 00100100
  - 00011000



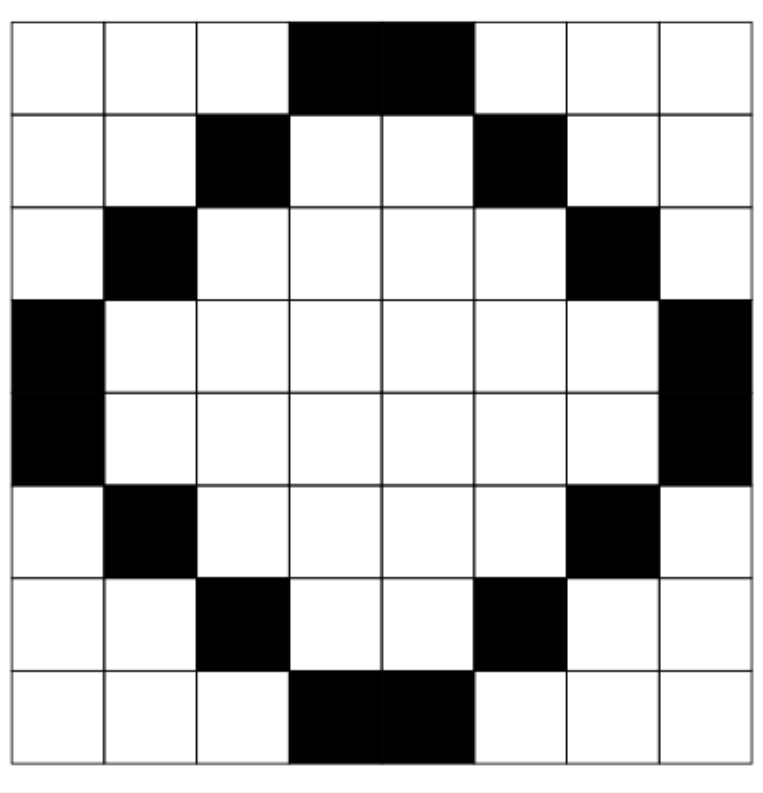
Consider the following

- How many zero-runs?
  - 00011000 3 0
  - 00100100 5 2
  - 01000010
  - 10000001
  - 10000001
  - 01000010
  - 00100100
  - 00011000



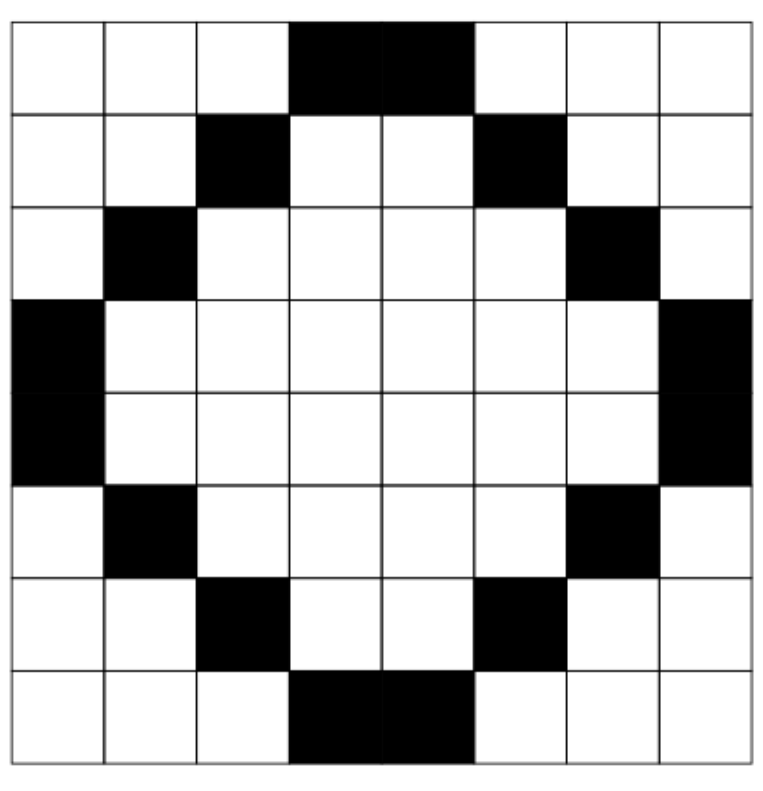
Consider the following

- How many zero-runs?
  - 00011000 3 0
  - 00100100 5 2
  - 01000010 3 4
  - 10000001
  - 10000001
  - 01000010
  - 00100100
  - 00011000



Consider the following

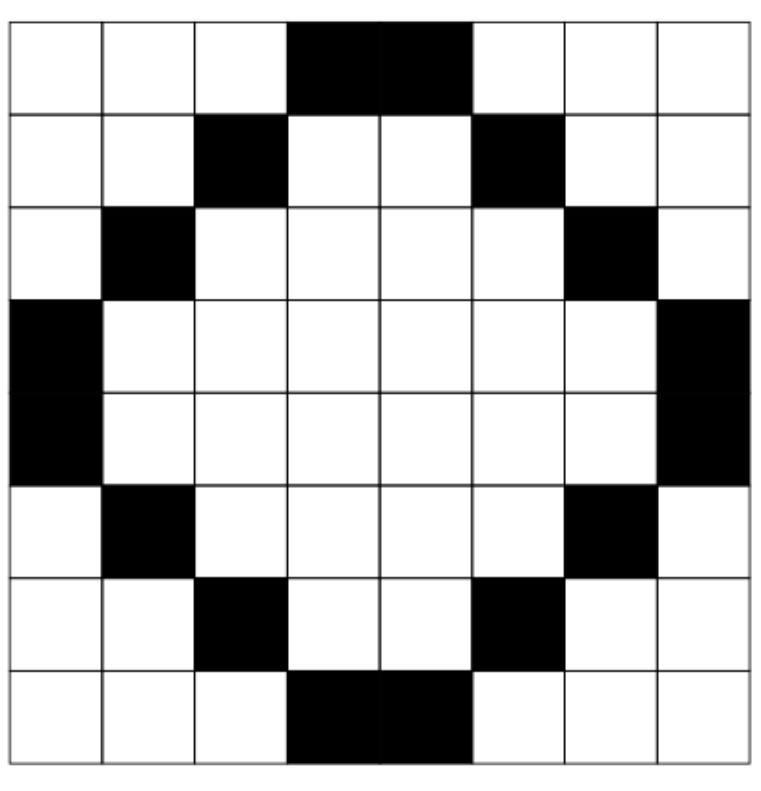
- How many zero-runs?
  - 00011000 3 0
  - 00100100 5 2
  - 01000010 3 4 1
  - 10000001 6
  - 10000001 0 6
  - 01000010 1 4
  - 00100100 3 2
  - 00011000 5 0 3



Converting encoding in binary

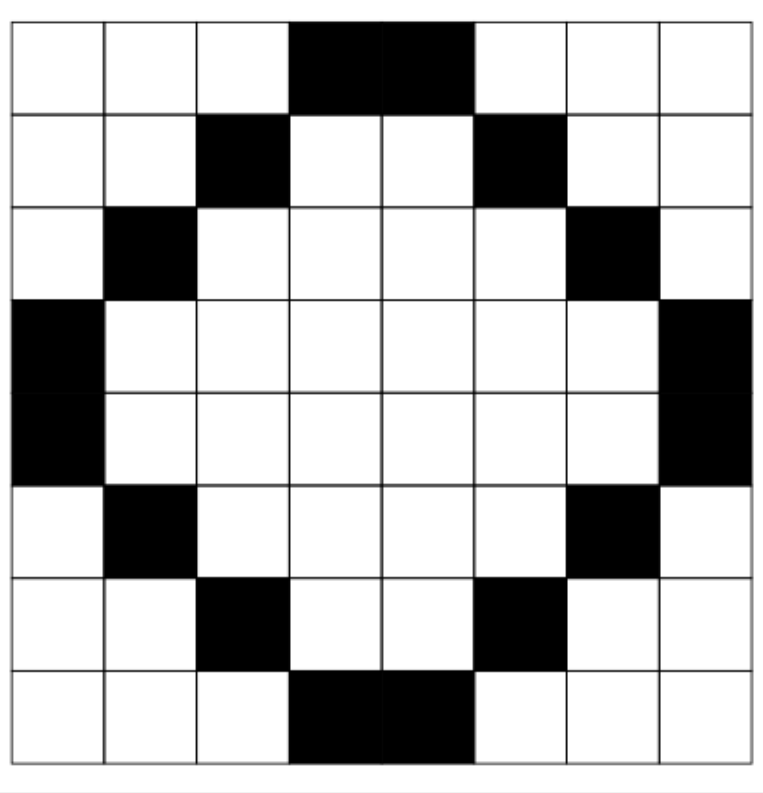
- 3,0,5,2,3,4,1,**6**,0,6,1,4,3,2,5,0,3

- Each of these seventeen values can be represented in 3 bits (because largest recorded number in the run is **6** and can be represented 110):



Converting encoding in binary

- From:
  00110000010010001000010100000011000000
  10100001000100100000011000 (64 bits)
- 3,0,5,2,3,4,1,6,0,6,1,4,3,2,5,0,3 (RLE)
- 011 000 101 010 011 100 001 110 000 110 001
  100 011 010 101 000 011 (17*3 = 51 bits)



Run-Length Encoding

- RLE saved us 10 bits (this is a very small image so we were never going to save much.)
- If we drew a similar picture at a resolution of 256 x 256 (say), we'd achieve much more impressive compression.
- **NOTE:** We would have to know that this encoding had been used, but provided we did, we could turn the 17 3-bit chunks back into the original pattern of 64 bits and display the image.

Run-Length: Lossless or Lossy?

- This technique is **lossless**.
- The exact data in the original file can be re-created from the information in the compressed file by reversing the process.
- No lossless compression algorithm can efficiently compress all possible data. For this reason, many different algorithms exist that are designed either with a specific type of input data in mind or with specific assumptions about what kinds of redundancy the uncompressed data are likely to contain.

Other forms of encoding?

- All compression schemes work by removing redundancy. Run-length encoding and Lempel-Ziv compression do this by reducing repetition.

- Another possible technique is to try to be more efficient about the encoding used for each data item. For example, rather than using ASCII for text (taking up one byte per character), can we use less? Perhaps by taking advantage of different *relative frequencies* of letters in the text. . . ?

Morse Code

- This idea was used by Samuel Morse when he invented his code for telegraphy.
- In this scheme, the letter ‘E’ is represented as a single dot, whereas ‘X’ is dash-dot-dot-dash. **Why?**
- Morse knew that E occurred in English more often than X, and tailored his code accordingly: common letters have shorter encodings, and less common ones have longer encodings.

Variable Length Bit String Encoding

- Remember that these values have an underlying bit string representation:
  - Text characters may be Extended ASCII (8 bits) or perhaps UTF-16

- **Observation**:

  The letter ‘e’ is far more common than the letter ‘q’

- **Idea**:

  Can we change the number of bits used to represent a value, making more frequent values use less bits than uncommon ones?

Values could be represented by **variable length bit strings**.

Variable Length Bit String Encoding

- To allow us to use variable bit-lengths, we need to know where one encoded value stops and the next begins.
- As long as we can ensure that *no encoding is a prefix of another*, then we can encode and decode these values safely.
- How can we ensure that property?

Huffman Tree

- One way to do this is by constructing a **binary tree**, and placing the characters on the leaves.
- The encoding is the path from root to leaf, and this means each character has a unique path with no other character as a prefix.
- This method is known as the **Huffman Tree.**

Huffman Tree Algorithm

- Start with the leaves containing the frequencies.
- While more than one node left do
  - Combine the two lowest frequency nodes into new subtree.
  - Remove those two nodes from consideration.
  - Add the root of the subtree with the combined frequency.

Example

Consider the text to encode:

CABAEACABCDEAEEFCEFF

Analyse the frequencies of each character:

Character	A	B	C	D	E	F
Frequency	5	2	4	1	5	3

Clearly A and E should have shortest encoding

Example

Method: Build the tree from the bottom up

Character	Count
A	5
B	2
C	4
D	1
E	5
F	3

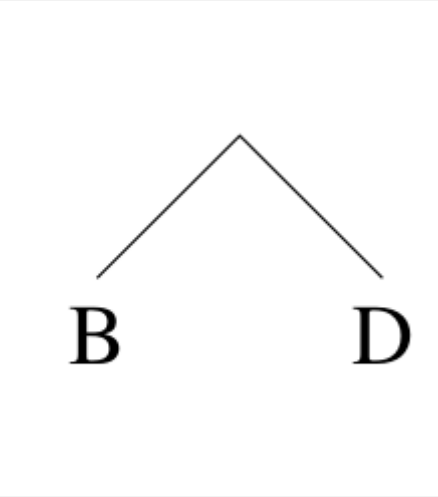
- Find 2 rows (sub-trees) with lowest frequency and join

Example

Method: Build the tree from the bottom up

Character	Count
A	5
B + D	3
C	4
E	5
F	3

- Find 2 rows (sub-trees) with lowest frequency and join

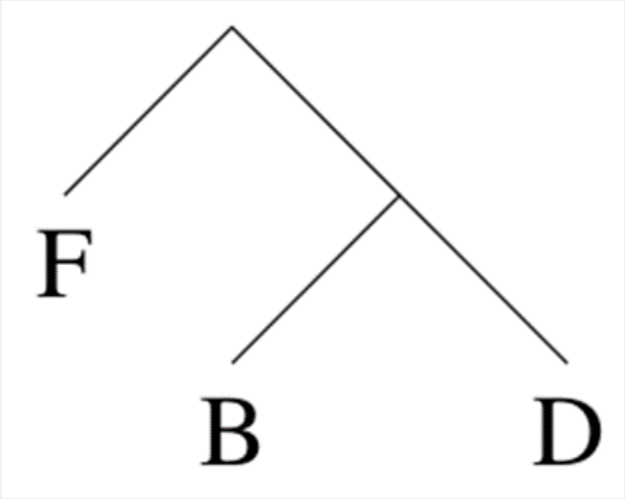


Example

Method: Build the tree from the bottom up

- Find 2 rows (sub-trees) with lowest frequency and join

Character	Count
A	5
F + (B + D)	6
C	4
E	5

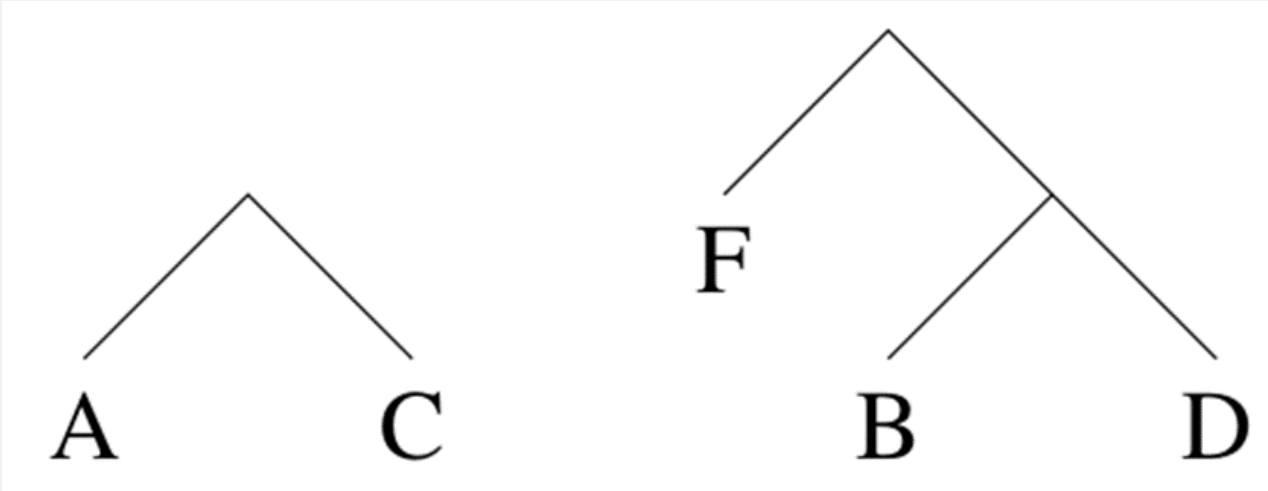


Example

Method: Build the tree from the bottom up

- Find 2 rows (sub-trees) with lowest frequency and join

Character	Count
A + C	9
F + (B + D)	6
E	5

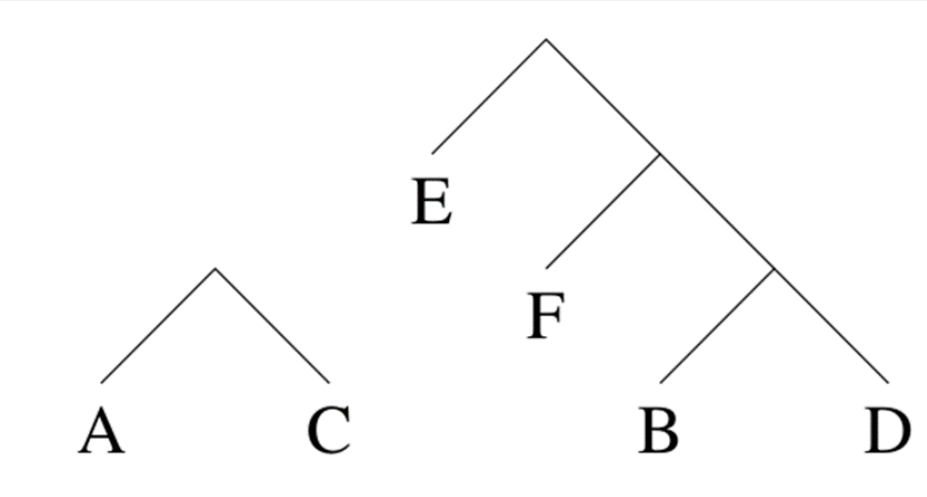


Example

Method: Build the tree from the bottom up

- Find 2 rows (sub-trees) with lowest frequency and join

Character	Count
A + C	9
E+ (F + (B + D))	11

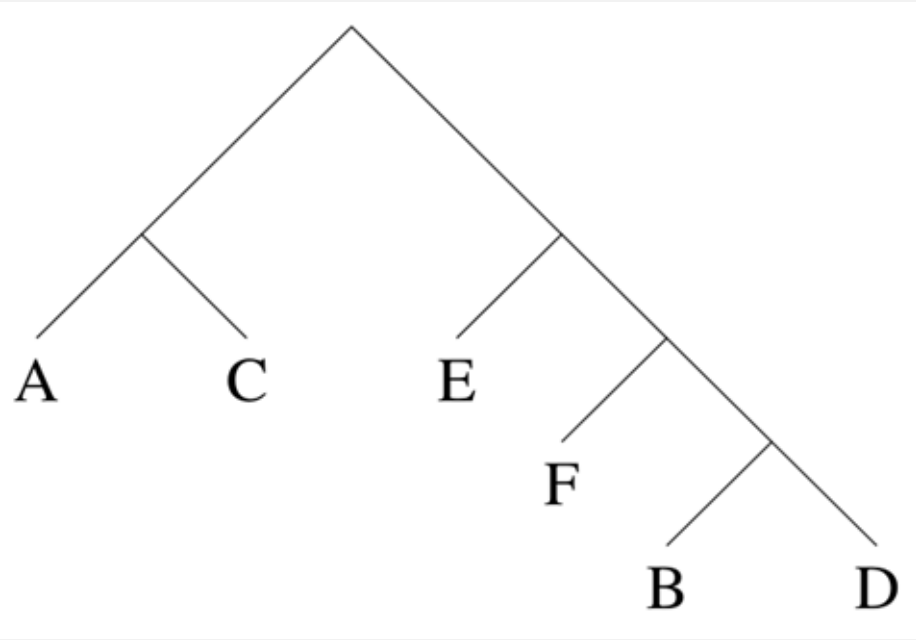


Example

Method: Build the tree from the bottom up

- Find 2 rows (sub-trees) with lowest frequency and join

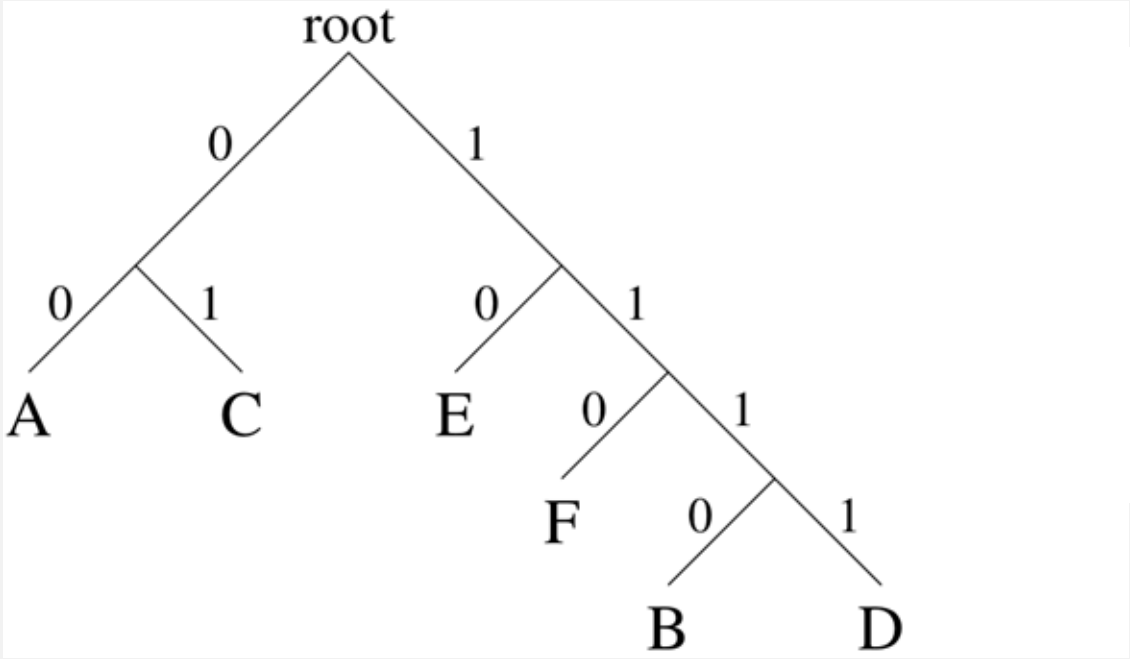
Character	Count
(A + C) + (E+ (F + (B + D)))	20



Example

- Now label **left** branches with 0 and **right** branches with 1

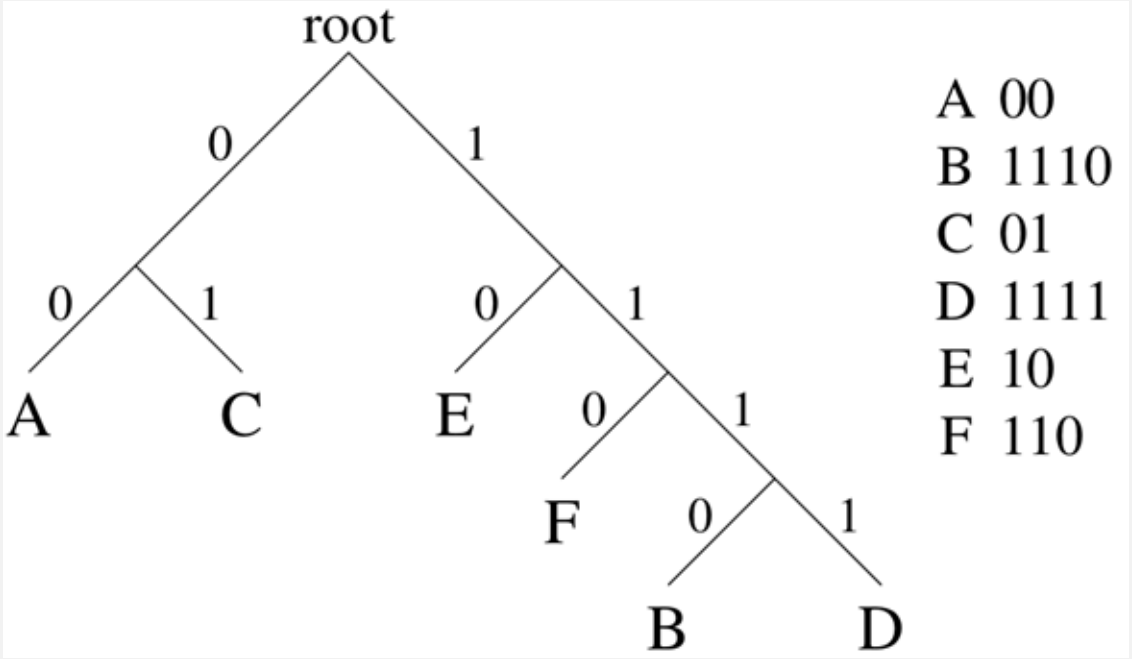
Character	Count
(A + C) + (E+ (F + (B + D)))	20



Example

Path from root to leaf gives bit string encoding for a given character

Character	Count
(A + C) + (E+ (F + (B + D)))	20



Example

Generate the encoded message by reading left to right, replacing the character with its mapped bit-string:

A	00
B	1110
C	01
D	1111
E	10
F	110

CABAEACABCDEAEEFCEFF

becomes

01001110001000010011100111111100010101100110110110

Variable vs Fixed Length

- The encoded message may look longer than the original.
- But remember that each character in the original message was represented with a fixed-length bit string.
  - E.g. If it was Extended-ASCII then it was 8 bits per character. Now it is less than 5 per character.
- The original message has 20 * 8 bits and the compressed has 49 bits. The compression rate is 49/160 = ~0.30 ☺
- Even better if it was in UTF-16: 49/320 = ~0.15!

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

0100111000100001001110011111110001010110011011011010

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

0100111000100001001110011111110001010110011011011110

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

01001110001000010011100111111100010101100110110110

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

C00111000100001001110011111100010101100110110110

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

C0011100010000100111001111111000101011001101101110

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

C00111000100001001110011111110001010110011011011010

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

CA1110001000010011100111111000101011001101101110

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

CA<u>I</u>1100010000100111001111111000101011001101101110

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

CA<u>11</u>1000100001001110011111100010101100110110110

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

CA<u>III</u>000100000100111001111111000101011001101101110

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

CA1110001000010011100111111000101011001101101110

Example

Decode:

- Read from left to right, checking for matches
- Swap match with mapped character
- Resume reading

A	00
B	1110
C	01
D	1111
E	10
F	110

CAB00100001001110011111110001010110011010110

Huffman codes are prefix codes

- The encoding produced is always a *prefix code*: no character's encoding occurs as a prefix in any other character's encoding.
- E.g. our encoding for C is 01; none of the other encodings start with 01.
  - 0100111000100001001110011111…
- Therefore we know that the 01 at the start means 'C'

Where else do we see prefix codes?

- Telephone area codes are similar:
  - London (large population) has the area code 020;
  - Leeds has 0113;
  - Cardiff has 029;
  - Newcastle has 01912
- Swansea has 01792.
  - 01790 – Spilsby
  - 01793 – Swindon
  - 0179 – No such prefix!
- No area code can prefix any other.

Could the encoded data ever be bigger than the unencoded data?

- Absolutely. If we encoded an image by encoding the runs of *ones* rather than zeroes, we'd end up using much more than 64 bits.

- The compression method must suit the data, although there are plenty of algorithms which should work well with any (uncompressed) data input. For small files it's often not worth it, however, particularly when you include the metadata necessary for decoding.

Is it worth it?

- It takes computation (and thus time and space) to compress data, and also to decompress it at the other end.

- It's ultimately a question of balancing requirements:
  - If storage and transmission are expensive (compared to the computation involved in compressing and decompressing), it's probably worth compressing;
  - if not, it might not be.

There's no simple answer, but a general rule of thumb is that there never seems to be enough storage space or bandwidth (usage expands to meet availability), so compression is often worthwhile.

Goals for Today

- Why we compress: storage and transmission
- Keyword and run-length encoding
- Lossless against lossy
- Variable-length codes and Huffman trees

CS-107 Lecture 8
Logic Gates and Tables

Troy Astarte

Megan Venn-Wycherley

Goals for Today

- Identify basic gates
- Observe gate behaviour via truth table, logic diagram, and Boolean expression
- Build circuits for logic gates

Gates and Circuits

- **Gates**

  A device that performs a basic operation on electrical signals.

- **Circuits**

  Gates combined to perform more complicated tasks.

Describing gates

Boolean expressions: Uses Boolean algebra, mathematical notation for expressing two-valued logic. Same algebra, but different symbols as CS-117.

Logic diagrams: A graphical representation of a circuit; each gate has its own symbol

Truth tables: A table showing all possible input values and the associated output values

Types of Logic Gates

Logic Gates

- Six types of gates
  - NOT
  - AND
  - OR
  - XOR
  - NAND
  - NOR

In CS-117 we don't consider XOR, NAND, and NOR as basic operations.

Likewise, from an electronics perspective, implication and equivalence are not basic gates

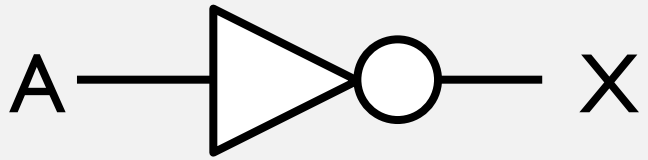
NOT

A NOT gate accepts one input signal (0 or 1) and returns the complementary (opposite) signal as output

Boolean Expression

$X = A'$

Logic Diagram



Truth Table

A	X
0	1
1	0

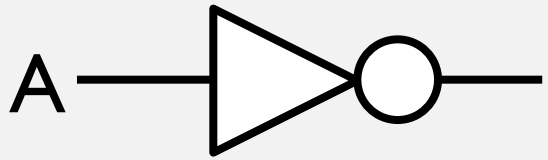
NOT in practice: a vacancy sign

- A car park sign shows VACANT when the bay sensor reads empty.
- S = 1 when a car is parked in the bay.
- **Vacant = NOT S**
- Car present: S = 1, so Vacant = 0. The sign stays off.
- Bay empty: S = 0, so Vacant = 1. The sign lights up.
- One input, and the output is always its opposite.

Your Turn - NOT

- Go to https://logic.ly/demo/ if you're on a laptop
- Truthtabletools.com if you're on mobile

- Can you create a NOT gate, with a switch and LED?



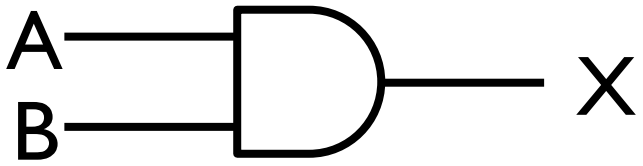
AND

An AND gate accepts two input signals. If both are 1, the output is 1; otherwise the output is 0.

Boolean Expression

$X = A \cdot B$

Logic Diagram



Truth Table

A	B	X
0	0	0
1	0	0
0	1	0
1	1	1

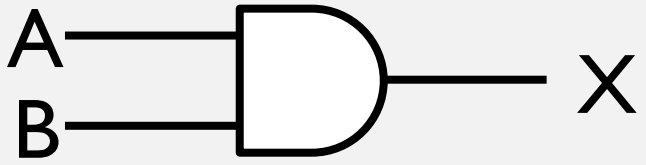
AND in practice: a microwave interlock

- A microwave should only heat when the door is shut and start is pressed.
- D = 1 when the door is closed, S = 1 when start is pressed.
- **Run = D · S**
- Door open, start pressed: 0 · 1 = 0, nothing happens.
- Door closed, nobody pressing: 1 · 0 = 0, still nothing.
- Door closed and start pressed: 1 · 1 = 1, it runs. Interlocks use AND because every condition has to hold.

Your Turn - AND

- Go to https://logic.ly/demo/ if you're on a laptop
- Truthtabletools.com if you're on mobile

- Can you create a AND gate, with two switches and LED?



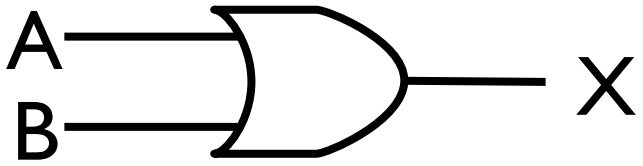
OR

An OR gate accepts two input signals. If both are 0, the output is 0; otherwise the output is 1.

Boolean Expression

$X = A + B$

Logic Diagram



Truth Table

A	B	X
0	0	0
1	0	1
0	1	1
1	1	1

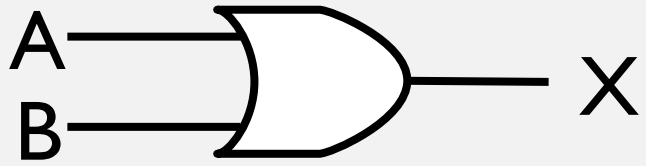
OR in practice: a car interior light

- A car's interior light comes on if either front door is opened.
- D = 1 when the driver's door is open, P = 1 when the passenger door is.
- **Light = D + P**
- Both shut: 0 + 0 = 0, the light stays off.
- Driver's door open: 1 + 0 = 1. Passenger door open: 0 + 1 = 1.
- Both open: 1 + 1 = 1. Any one condition is enough, so OR never needs all of them.

Your Turn - OR

- Go to https://logic.ly/demo/ if you're on a laptop
- Truthtabletools.com if you're on mobile

- Can you create a OR gate, with two switches and LED?



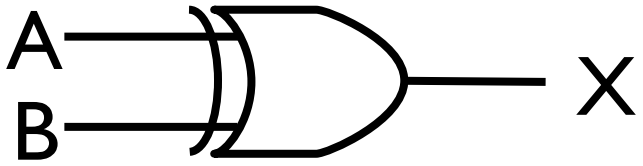
XOR

An XOR gate accepts two input signals. If both are the same, the output is 0; otherwise, the output is 1

Boolean Expression

$$X = A \oplus B$$

Logic Diagram



Truth Table

A	B	X
0	0	0
1	0	1
0	1	1
1	1	0

XOR in practice: two switches, one light

- A landing light has a switch at the bottom and top of the stairs.
- A and B are 1 when that switch is up. The light is on when they disagree.
- **Light = A ⊕ B**
- Both down: 0 ⊕ 0 = 0, off. Both up: 1 ⊕ 1 = 0, off again.
- One up, one down: 1 ⊕ 0 = 1, on.
- Flipping either switch always changes the light, which is how two-way switching works.

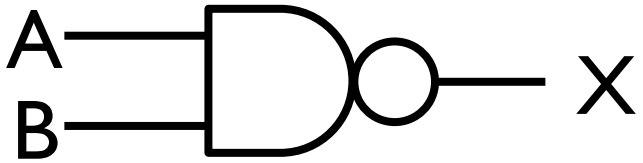
NAND

A NAND (“NOT of AND”) gate accepts two input signals. If both are 1, the output is 0; otherwise, the output is 1.

Boolean Expression

$X = (A \cdot B)'$

Logic Diagram



Truth Table

A	B	X
0	0	1
1	0	1
0	1	1
1	1	0

NAND in practice: level crossing barriers

- A level crossing has two barriers. A red warning light shows until both are confirmed down.
- B1 = 1 when the near barrier is fully down, B2 = 1 when the far one is.
- **Warn = (B1 · B2)' The light is lit while Warn = 1.**
- Neither down: (0 · 0)' = 1, light on.
- Near barrier only: (1 · 0)' = 1, light on.
- Far barrier only: (0 · 1)' = 1, light on.
- Both down: (1 · 1)' = 0, light off and the train may proceed.
- NAND drives the warning high for every state except the one safe combination.

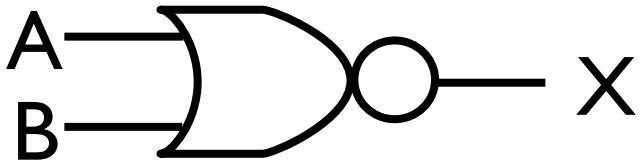
NOR

The NOR (“NOT of OR”) gate accepts two inputs. If both are 0, the output is 1; otherwise, the output is 0.

Boolean Expression

$$X = (A + B)'$$

Logic Diagram



Truth Table

A	B	X
0	0	1
1	0	0
0	1	0
1	1	0

NOR in practice: a lift door interlock

- A lift may only move when no door sensor sees an obstruction.
- S1 and S2 are 1 when that sensor is blocked.
- **Safe = (S1 + S2)'**
- Nothing blocking: (0 + 0)' = 1, the lift can move.
- One sensor blocked: (1 + 0)' = 0, it stays put.
- Both blocked: (1 + 1)' = 0. NOR gives 1 only when every input is 0, so it is the "all clear" gate.

A note on notation

Here we have seen the use of **+**, **•**, and **′** - You may use **∨**, **∧** , and **¬** from propositional logic.

You may prefer the words **OR**, **AND**, and **NOT**, or even **disjunction**, **conjunction**, and **negation**.

You may even be familiar with **~** or **!** for negation.

Just don't mix them. Stick to a convention.

Building Logic Gates

Building gates: from symbols to circuits

- **A real gate is a small electrical circuit** that behaves exactly the way that table says.
- **Near the supply voltage counts as 1.** Near zero volts counts as 0.
- **Typical logic:** 3.3V or 5V for a 1, and 0V, called Ground, for a 0.

Why only two levels?

- Wires pick up interference, and components drift with heat, age and manufacturing tolerance and two levels at opposite ends of the range stay distinguishable even when the voltage sags by a volt.

- Ten levels in the same range would sit a few tenths of a volt apart and be misread constantly, so binary is an engineering decision about reliability! It also makes each component simple: a switch is either on (1) or off, (0) and that is all it has to get right.

Constructing gates

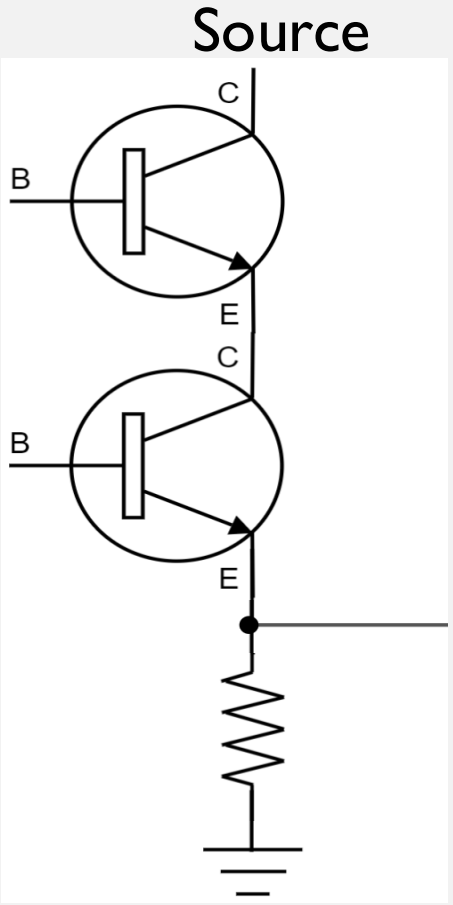
- Device that acts either as a **wire that conducts electricity** or as **a resistor that blocks the flow of electricity**, depending on the **voltage level of an input** signal.

- A **transistor** has no moving parts, yet it acts like a switch. Made of a **semiconductor** material, which is neither a particularly good conductor of electricity nor a particularly good insulator. They are the basic building blocks for gates.

- **Resistors** prevent transistors connecting source directly to the ground, burning them out.

How to read a circuit diagram

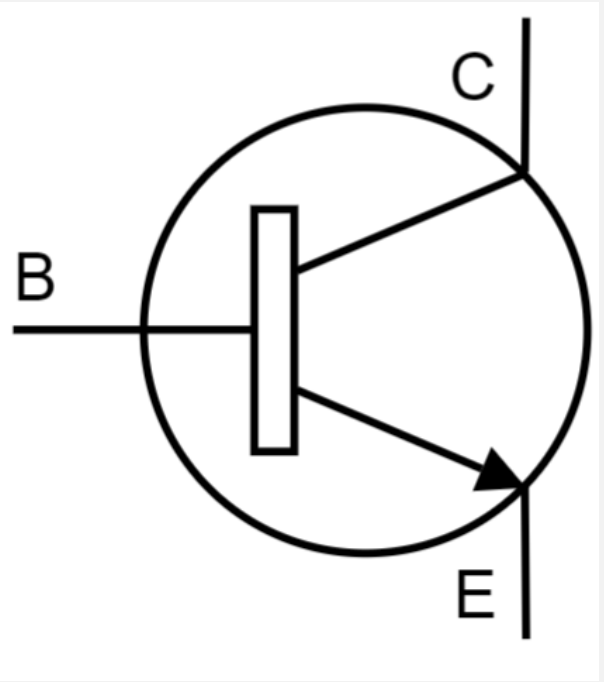
- **Source, at the top:** the supply voltage, permanently a 1.
- **Ground, the striped symbol at the bottom:** permanently a 0.
- **Lines are wires.** A dot where they meet means they are joined; crossing without a dot means they are not.
- Circles are our transistors, zigzag is a resistor.



Constructing gates

- A transistor has three terminals:
  - A **B**ase
  - A **C**ollector
  - An **E**mitter

- If current flows into the **Emitter** then this results in the **Source** being connected to the **Ground**. This causes the **output voltage to drop**.

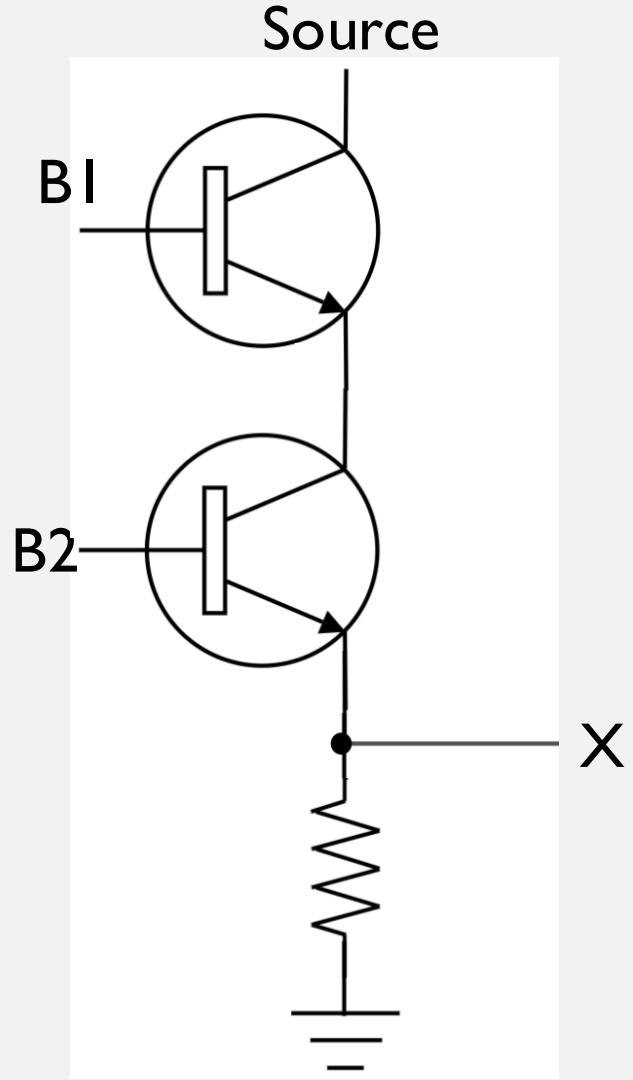


Two switches in series make AND

- **Wire switch A and switch B one after the other** between the supply and a lamp.
- **There is only one path,** and it runs through both switches.
- **A closed, B open:** no path, lamp off. A open, B closed: no path either.
- **Both closed:** current flows and the lamp lights.
- **Lamp = A · B.** That is an AND gate, built from two switches and no electronics at all.

Constructing AND gates

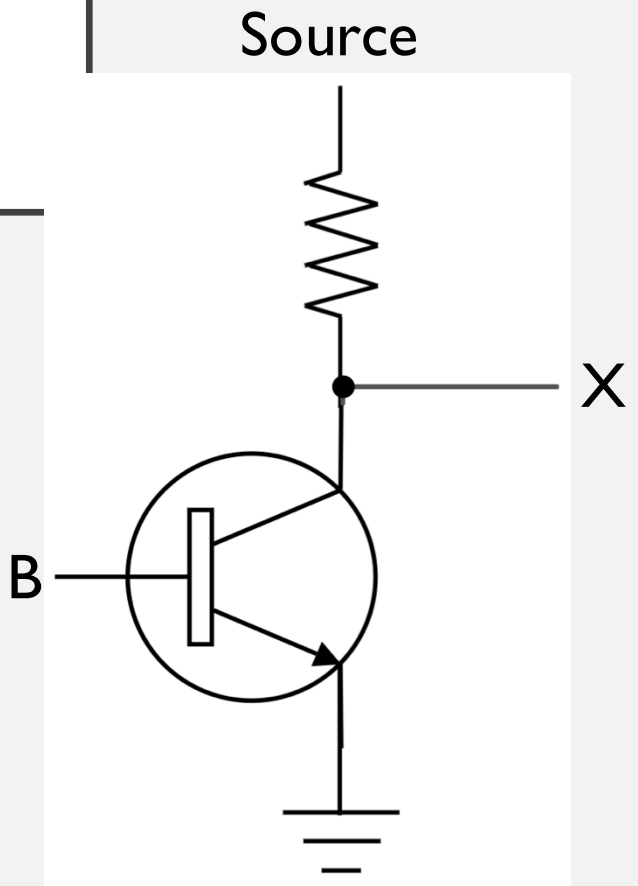
- **Two NPN transistors in series,** stacked between the Source and Ground.
- **B1 and B2 feed the two bases.** A high signal on a base lets that transistor conduct.
- **Output X** is tapped below the pair, above the resistor to Ground.
- **Both high:** both conduct, the Source reaches X, output 1.
- **Either low:** the chain breaks, X is pulled to Ground, output 0.



$$X = B1 \cdot B2$$

Constructing NOT gates

- This diagram shows how an NPN transistor might be connected to give a NOT gate.
- If there is a **high signal** coming into the base of the transistor, then the transistor lets **current flow through**. Thus pulling the out **signal low**.
- If there is a **low signal** coming into the base of the transistor, then the transistor does **not let any current through**. Thus, allowing the out **signal high**.



X = B'

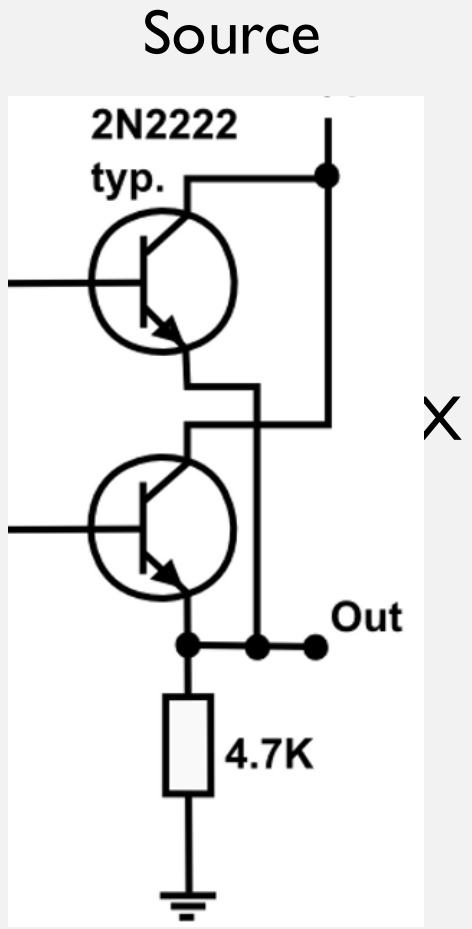
Two switches in parallel make OR

- Now wire the two switches side by side, both bridging the same gap in the circuit. There are two possible paths, and the current only needs one of them.

- Either switch closed: the lamp lights. Both closed: it stays lit. Both open: no path at all, lamp off.

Constructing OR gates

- **Two NPN transistors in parallel,** both fed from the Source, their outputs joined at Out.
- **B1 and B2 feed the two bases.** A high signal on either base lets that transistor conduct.
- **Either input high:** that transistor connects the Source to Out, so the output is 1.
- **Both inputs low:** neither conducts, and the 4.7K resistor holds Out down at Ground, so the output is 0.
- **Parallel means OR.** Current needs only one path, so one high input is enough. In series it would be AND.



X

X = B1 + B2

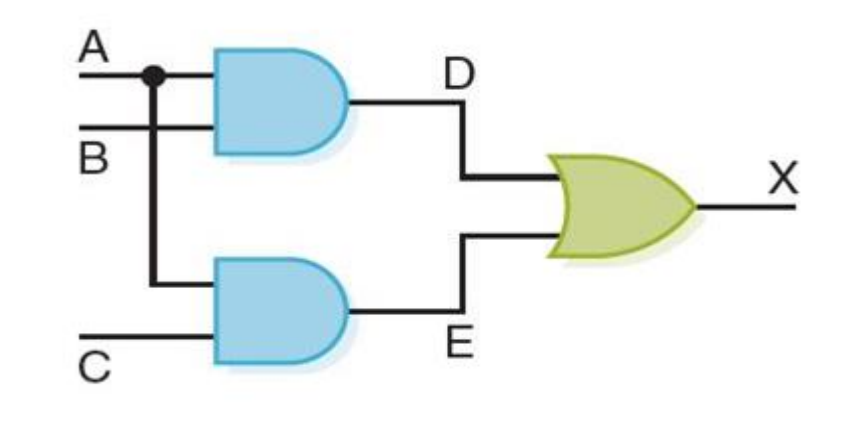
Circuits

- We can combine individual gates together into more complex **circuits**

- Circuits can be described by:
  - Boolean expressions: Same as for gates.
  - Truth tables: Same as for gates.
  - Logic diagrams: A graphical representation combining gate symbols.

Combinational Circuits

- Gates are combined into circuits by using the output of one gate as the input for another.



$$A \cdot B + A \cdot C$$

A	B	C	X
0	0	0	0
I	0	0	0
0	I	0	0
I	I	0	I
0	0	I	0
I	0	I	I
0	I	I	0
I	I	I	I

Goals for Today

- Identify basic gates
- Observe gate behaviour via truth table, logic diagram, and Boolean expression
- Build circuits for logic gates

CS-107 Lecture 8
Logic Gates and Tables

Troy Astarte

Megan Venn-Wycherley